AI-Powered CLI for Rocket.Chat App Generation

Google Summer of Code Proposal

By: Revansh Sharma
Mentors: Dnouv, Gabriel Casals

Abstract

In the evolving landscape of software development, the need for efficiency and accessibility is paramount. This proposal outlines the creation of an innovative AI-powered CLI tool poised to redefine the development process for Rocket.Chat applications. By leveraging natural language prompts and augmenting existing tools such as gemini-cli, this system will seamlessly incorporate the comprehensive knowledge of the Rocket.Chat Apps Engine, automating the entire development lifecycle.

The objective is to streamline the app development process, effectively eliminating redundant boilerplate code, and enabling both novice and veteran developers to swiftly build production-ready applications with ease.

This tool has the potential to significantly expand the Rocket.Chat app ecosystem by lowering the barrier to entry for developers.

Problem Statement

The current Rocket.Chat app development environment presents several challenges:

- Navigating the intricate architecture of the Apps Engine
- The repetitive and cumbersome nature of boilerplate code
- Manual integration of APIs, including webhooks and UI components
- Labor-intensive testing and deployment processes

These hurdles lead to:

- A steep learning curve for beginners
- Lengthy development cycles
- Increased risk of errors
- Restricted rapid prototyping capabilities

At present, there is no intelligent tool that abstracts these complexities while preserving flexibility.

Proposed Solution

Introducing an AI-driven CLI tool that transforms user prompts into fully functional Rocket.Chat applications.

Key features of the solution include:

- Comprehensive app scaffolding generation
- Dynamic feature addition (e.g., commands, APIs, UI components)
- Continuous maintenance and updating of test cases
- Facilitated deployment processes

This innovative tool shifts the development paradigm from:

Manual Coding → AI-Assisted Development

System Architecture

Architecture Flow:

1. **User Prompt**
2. **CLI Parser**
3. **Context-Aware Prompt Builder**
4. **LLM Engine (Gemini / OpenAI)**
5. **Code Generator (Templates + AI Output)**
6. **File System Writer**
7. **Rocket.Chat App**

Core Components:

1. **CLI Layer:** Manages user commands, parses intent, and arguments.

2. **AI Engine:** Utilizes prompt engineering and templates to convert prompts into structured outputs.
3. **Knowledge Base:** Integrates Rocket.Chat Apps Engine structure, API schemas, and best practices.
4. **Code Generator:** Dynamically produces TypeScript code, updating project files as necessary.
5. **Testing Engine:** Automatically generates Jest test cases to ensure robust test coverage.

CLI Design & Workflow

Example Commands:

- rc-app-gen create my-app
- rc-app-gen add slash-command hello
- rc-app-gen add webhook
- rc-app-gen add persistence
- rc-app-gen test
- rc-app-gen deploy

Internal Flow:
Command → Intent Detection → Prompt Construction → AI → Code Generation → File Injection

---

AI & Prompt Engineering

Structured prompts are essential for ensuring consistent and reliable output.

Example:

User Input:
Create a slash command that greets the user.

Generated Prompt:
Generate a Rocket.Chat app using Apps Engine.
Requirements:

- Use TypeScript
- Implement a slash command
- Adhere to Apps Engine interfaces
- Return a greeting message

Output:
Complete class implementation.

Technical Implementation

Overview

The system will be implemented as a modular CLI tool using Node.js and TypeScript. It follows a **layered architecture** to separate command handling, AI processing, code generation, and file management.

The implementation ensures:

- Maintainability
- Extensibility
- Clean separation of concerns

Tech Stack

Language: TypeScript

Runtime: Node.js

CLI Framework: Commander.js

AI Integration: REST APIs (Gemini / OpenAI)

Testing: Jest

Linting: ESLint + Prettier

Project Structure (GitHub-Ready)

```
rc-app-gen/
│
├── src/
│   ├── cli/            # CLI command definitions
│   │   ├── index.ts      # Entry point for CLI
│   │   ├── commands/
│   │   │   ├── create.ts
│   │   │   ├── add.ts
│   │   │   ├── test.ts
│   │   │   └── deploy.ts
│   │
```

```
│   ├── core/          # Core logic of the system
│   │   ├── promptBuilder.ts
│   │   ├── aiClient.ts
│   │   ├── codeGenerator.ts
│   │   ├── validator.ts
│   │   └── fileManager.ts
│   │
│   ├── features/      # Feature modules
│   │   ├── slashCommand/
│   │   │   ├── template.ts
│   │   │   └── generator.ts
│   │   ├── webhook/
│   │   ├── persistence/
│   │   ├── scheduler/
│   │   └── ui/
│   │
│   ├── templates/     # Base app templates
│   │   ├── base-app/
│   │   │   ├── app.ts
│   │   │   ├── package.json
│   │   │   └── tsconfig.json
│   │
│   ├── utils/         # Helper utilities
│   │   ├── logger.ts
│   │   ├── formatter.ts
│   │   └── constants.ts
```

```
│   │
│   └── types/          # Type definitions
│       └── index.ts
│
├── tests/              # Test suite
│   ├── unit/
│   ├── integration/
│   └── mocks/
│
├── examples/           # Example generated apps
│
├── docs/               # Documentation
│
├── package.json
├── tsconfig.json
├── .eslintrc
├── .prettierrc
└── README.md
```

Execution Flow

```
User Command
  ↓
CLI (Commander.js)
  ↓
Command Handler (create / add / test / deploy)
  ↓
Prompt Builder (adds context + templates)
```

↓

AI Client (API call)

↓

Code Generator (merge AI + templates)

↓

Validator (TS + lint checks)

↓

File Manager (write/update files)

Core Modules

1. CLI Layer (/cli)

- Handles user commands
- Routes commands to appropriate handlers

2. Prompt Builder (core/promptBuilder.ts)

- Converts user input into structured prompts
- Injects Rocket.Chat context

3. AI Client (core/aiClient.ts)

- Handles API calls to LLMs
- Supports multiple providers (Gemini/OpenAI)

4. Code Generator (core/codeGenerator.ts)

- Combines:
  - Templates
  - AI output
- Produces valid TypeScript files

5. Validator (core/validator.ts)

- Ensures generated code is:
  - Type-safe
  - Lint-compliant
- Rejects invalid outputs

6. File Manager (core/fileManager.ts)

- Handles:

- File creation
- Updates
- Directory structure

Feature Module Design

Each feature follows a modular pattern:

```
feature-name/
├── template.ts     # Base structure
├── generator.ts    # AI + template logic
```

This ensures:

- Easy extension
- Plug-and-play features
- Clean separation

Testing Strategy

```
tests/
├── unit/          # Core module tests
├── integration/   # CLI + feature tests
└── mocks/         # Mock Rocket.Chat APIs
```

Validation Layer

Before writing files:

- TypeScript compilation check
- ESLint validation
- Schema validation for Rocket.Chat APIs

Invalid outputs are:

Rejected → Regenerated → Revalidated

Example Generated Code

```
export class HelloCommand implements ICommand {
  public command = 'hello';

  public async executor(context, read, modify) {
    const message = modify.getCreator().startMessage()
      .setText('Hello from AI-generated app!');
    return modify.getCreator().finish(message);
  }
}
```

Feature Modules

The CLI will facilitate modular feature generation, including:

- Slash Commands
- Webhooks / External APIs
- Persistence (Database storage)
- User-specific state management
- Scheduled jobs
- UI Kit components (buttons, modals)

Each module will be dynamically generated and integrated to ensure seamless development.

UI Generation

UI components will be crafted using Rocket.Chat UIKit, ensuring:

- Consistent and intuitive UX
- Responsive design
- Native Rocket.Chat compatibility

Testing Strategy

Automated generation of:

- Unit tests using Jest
- Mock Rocket.Chat APIs
- Optional integration tests

Ensuring:

- Code reliability
- Maintainability
- Production readiness

Evaluation Metrics

Project success will be measured by:

- App generation time (< 30 seconds)
- Percentage of functional generated apps
- Test coverage (> 80%)
- Deployment success rate

Risks & Mitigation

Risks will be actively mitigated through a hybrid approach combining template-based generation, validation layers, and iterative regeneration of invalid outputs, ensuring reliability and consistency.

Final Deliverables

- Fully functional AI-powered CLI tool

- Support for core Rocket.Chat features (commands, webhooks, persistence)

- Automated test generation system

- Documentation and usage guide

- Example generated applications

Timeline (90 Hours)

- **Phase 1 (0–20 hrs):** Research and CLI setup
- **Phase 2 (20–50 hrs):** Core CLI and AI integration
- **Phase 3 (50–75 hrs):** Feature modules implementation
- **Phase 4 (75–85 hrs):** Testing system development
- **Phase 5 (85–90 hrs):** Documentation and final polishing

About Me

I am Revansh Sharma, an aspiring developer focused on building AI-driven systems and developer tools that solve real-world problems.

I actively work on projects at the intersection of AI, software engineering, and product development. My recent work includes developing AI-based solutions, experimenting with

LLM integrations, and building tools that improve developer productivity.

Currently, I am also working on my own game project “CTRL : REFLƎX”, where I am designing systems involving open-world mechanics, NPC behavior, and interactive environments using modern tools like Unreal Engine and Blender. This has strengthened my ability to think in terms of system architecture, scalability, and user experience.

Alongside this, I am building and exploring AI-powered solutions using platforms like Replit and working towards monetizing small AI tools and services through freelancing. I have experience with TypeScript, modern JavaScript ecosystems, and integrating APIs to build functional applications and I have been working on open source tools such as “KrishiMitra” to empower farmers.

I also run “ZeroTheory”, where I document my journey of going from zero to building impactful tech projects. This reflects my mindset of continuous learning, experimentation, and execution.

My core interest lies in simplifying complex systems and making technology more accessible. I am particularly drawn to developer tooling, where small innovations can significantly improve productivity and learning curves.

This project aligns perfectly with my current direction, as I am already working with AI systems and exploring ways to apply them to real development workflows. I am confident in my ability to not only complete this project but also deliver a tool that provides real value to the Rocket.Chat ecosystem.

I am committed to open-source contribution and eager to collaborate closely with mentors to deliver a high-quality, production-ready solution.

Impact on Rocket.Chat Ecosystem

- Enables more developers to build apps
- Increases number of integrations
- Accelerates community contributions
- Reduces onboarding friction

This tool can significantly expand the Rocket.Chat app ecosystem.

Future Scope

- Development of a GUI-based app builder
- Establishment of a plugin ecosystem

- Expansion to multi-platform support (Slack, Discord)
- Advanced AI-assisted debugging capabilities

Conclusion

This project is set to revolutionize Rocket.Chat app development, transforming it into a rapid, intelligent, and accessible process. By combining AI with CLI tooling, it promises:

- Accelerated development timelines
- Reduced complexity
- Enhanced accessibility for developers of all skill levels

I am confident in delivering a complete, production-ready solution within the given timeline.

Thank You Note

Thank you so much for taking the time to thoroughly review my proposal. Your feedback and consideration are greatly appreciated.